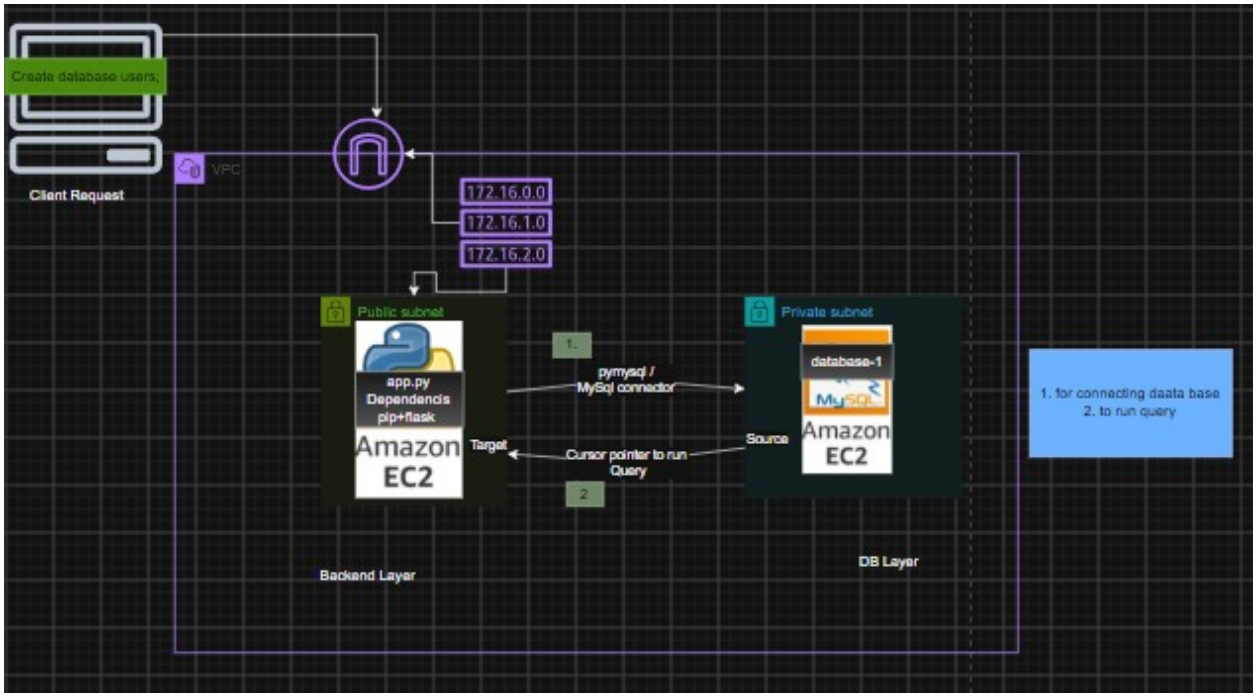
Connecting DB using Python Backend

Task-

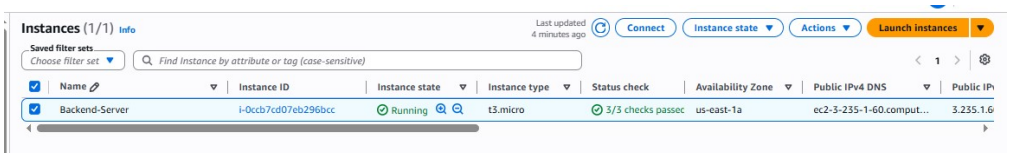


Prerequisite:-

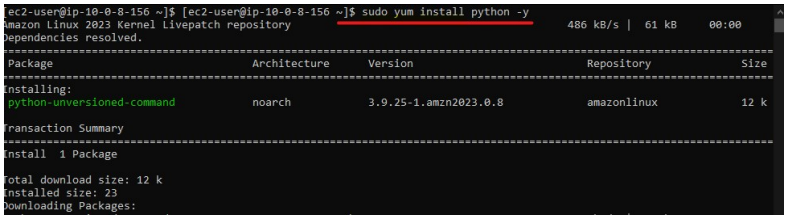
1. one **Backend server**(in this case a - Bastion Host).
    - Install Python and its dependencies(pip +flask +pymysql or Mysql connector + flask_cors)
    - Create **app.py** to store appliaction code.
2. one AWS RDS database in private subnet(as of now give public access).
3. MySql workbench for testing only ,in real time will use frontend layer.
4. Now configure app.py in Backend-server (Bastion) to communicate with RDS database-1 .
5. Then run app.py application and fetch DB details .

Steps:

1. Create Bastion Host and configure python for backend.

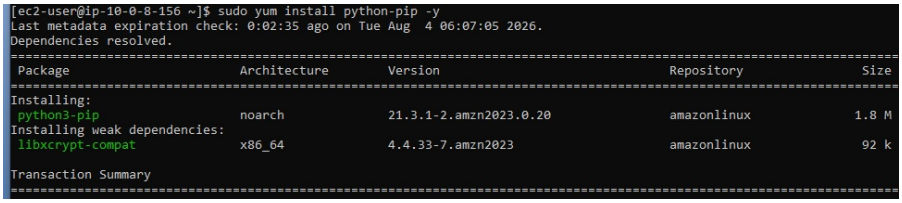


a. Install python $**sudo yum install python –y**

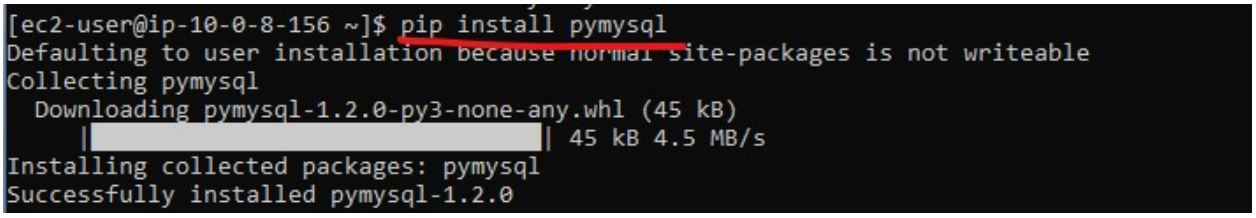


You can check if its installed or not by $python --version

b. Install package manager **$sudo yum install python-pip –y**

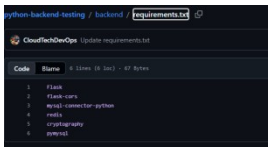


c. Install **$ `pip install pymysql`**

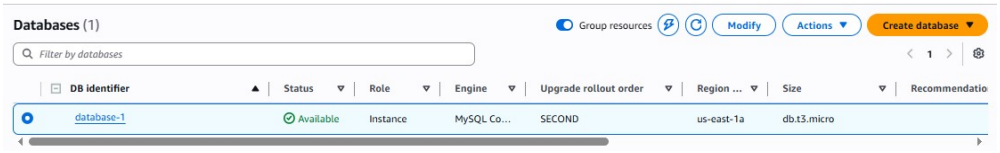


d. Same way install flask , flask_cors and **`mysql-connector-python`(no required if we have installed `pymysql`)**

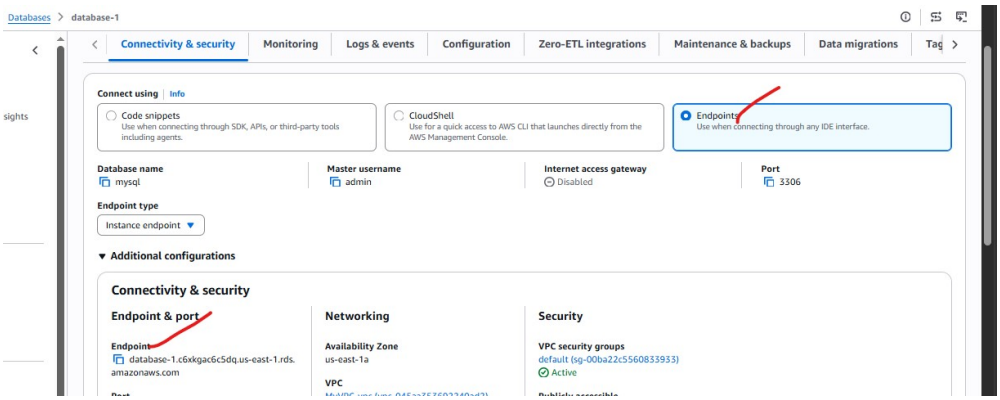
[instead of installing individual package , we can mention packages inside ***requirements.txt*** , to install all the packages at a time] here we have not implement this its for knowledge.as of you can ignore



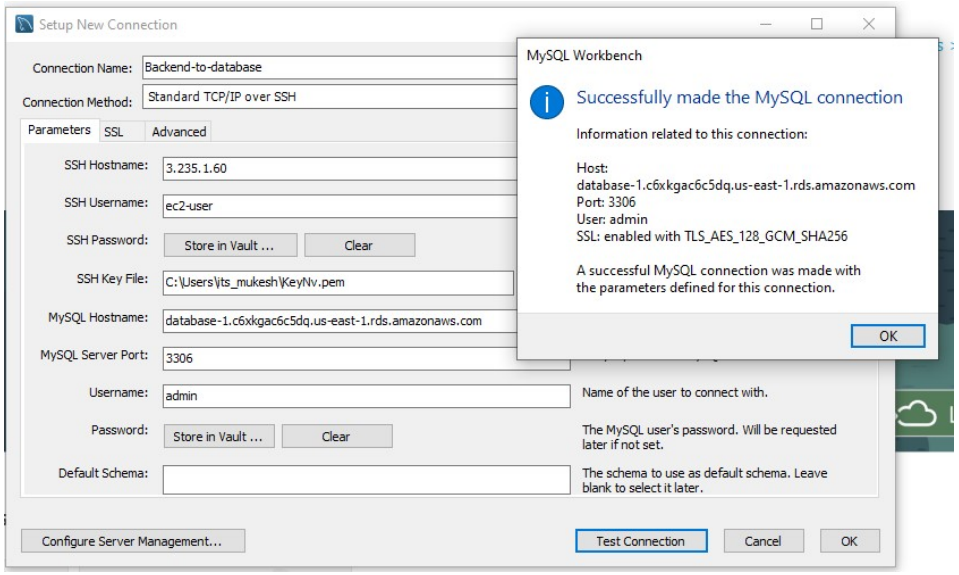
2. Create one AWS RDS database in private subnet(as of now give public access).
   a. Create a public database



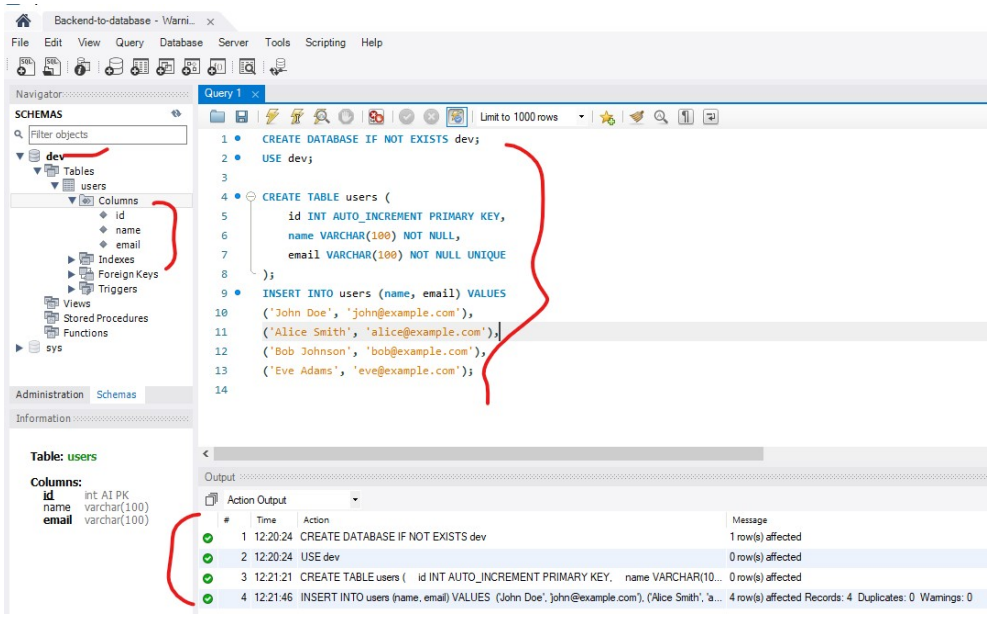
   b. Note the **endpoint dns** > database-1.c6xkgac6c5dq.us-east-1.rds.amazonaws.com



3. Now Test the connection to RDS database-1 via MySql Workbench.



Below you can see we were able to communicate with DB using MYSQL workbench in our local system.



4. Now configure app.py to communicate with RDS database-1

Login into Backend-server (Bastion host) > open **app.py** file > paste below backend code.

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import mysql.connector
from mysql.connector import Error

app = Flask(__name__)
CORS(app) #enable CORS for all routes

# =========================
# DATABASE CONFIG
# =========================

# ✍️ Writer (Master)
db_write_config = {
    'host': 'database-1.c6xkgac6c5dq.us-east-1.rds.amazonaws.com',
    'user': 'admin',
    'password': 'AdminMukesh',
    'database': 'dev'
}

# 📖 Reader (Read Replica Endpoint)
db_read_config = {
    'host': 'database-1.c6xkgac6c5dq.us-east-1.rds.amazonaws.com',
    'user': 'admin',
    'password': 'AdminMukesh',
    'database': 'dev'
}

# =========================
# CONNECTION HELPERS
# =========================

def get_write_connection():
    return mysql.connector.connect(**db_write_config)

def get_read_connection():
    return mysql.connector.connect(**db_read_config)

def get_db_info(cursor):
    cursor.execute("SELECT @@hostname AS host, @@read_only AS read_only")
    return cursor.fetchone()

# =========================
# ROUTES
# =========================

@app.route('/')
def index():
    return "API is running"
```

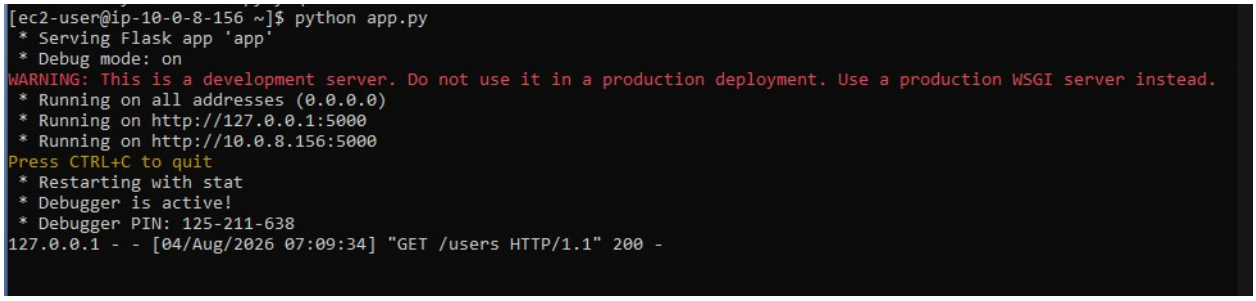
```python
# ========================
# READ APIs (Reader DB)
# ========================

@app.route('/users', methods=['GET'])
def get_users():
    try:
        conn = get_read_connection()
        cursor = conn.cursor(dictionary=True)

        cursor.execute("SELECT * FROM users")
        users = cursor.fetchall()

        db_info = get_db_info(cursor)

        return jsonify({
            "data": users,
            "served_by": db_info["host"],
            "read_only": db_info["read_only"],
            "db_role": "READER" if db_info["read_only"] == 1 else "WRITER"
        })
    except Error as err:
        return jsonify({'error': str(err)}), 500
    finally:
        cursor.close()
        conn.close()


@app.route('/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    try:
        conn = get_read_connection()
        cursor = conn.cursor(dictionary=True)

        cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
        user = cursor.fetchone()

        if not user:
            return jsonify({'error': 'User not found'}), 404

        db_info = get_db_info(cursor)

        return jsonify({
            "data": user,
            "served_by": db_info["host"],
            "read_only": db_info["read_only"],
            "db_role": "READER" if db_info["read_only"] == 1 else "WRITER"
        })
    except Error as err:
        return jsonify({'error': str(err)}), 500
    finally:
        cursor.close()
        conn.close()

# ========================
# WRITE APIs (Master DB)
# ========================

@app.route('/users/add', methods=['POST'])
```

```python
def add_user():
    data = request.json
    name = data.get('name')
    email = data.get('email')

    if not name or not email:
        return jsonify({'error': 'Name and Email are required'}), 400

    try:
        conn = get_write_connection()
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO users (name, email) VALUES (%s, %s)",
            (name, email)
        )
        conn.commit()
        return jsonify({'message': 'User added successfully'}), 201
    except Error as err:
        return jsonify({'error': str(err)}), 500
    finally:
        cursor.close()
        conn.close()


@app.route('/users/update/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    data = request.json
    name = data.get('name')
    email = data.get('email')

    if not name or not email:
        return jsonify({'error': 'Name and Email are required'}), 400

    try:
        conn = get_write_connection()
        cursor = conn.cursor()

        cursor.execute("SELECT id FROM users WHERE id = %s", (user_id,))
        if not cursor.fetchone():
            return jsonify({'error': 'User not found'}), 404

        cursor.execute(
            "UPDATE users SET name = %s, email = %s WHERE id = %s",
            (name, email, user_id)
        )
        conn.commit()
        return jsonify({'message': 'User updated successfully'})
    except Error as err:
        return jsonify({'error': str(err)}), 500
    finally:
        cursor.close()
        conn.close()


@app.route('/users/delete/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    try:
        conn = get_write_connection()
```

```
        cursor = conn.cursor()

        cursor.execute("SELECT id FROM users WHERE id = %s", (user_id,))
        if not cursor.fetchone():
            return jsonify({'error': 'User not found'}), 404

        cursor.execute("DELETE FROM users WHERE id = %s", (user_id,))
        conn.commit()
        return jsonify({'message': 'User deleted successfully'})
    except Error as err:
        return jsonify({'error': str(err)}), 500
    finally:
        cursor.close()
        conn.close()

# ========================
# ENTRY POINT
# ========================
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

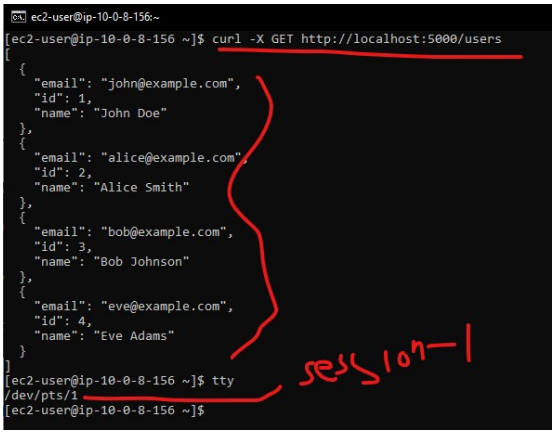
5. Then run app.py application and fetch DB details .

   a. Run app.py **$python app.py**

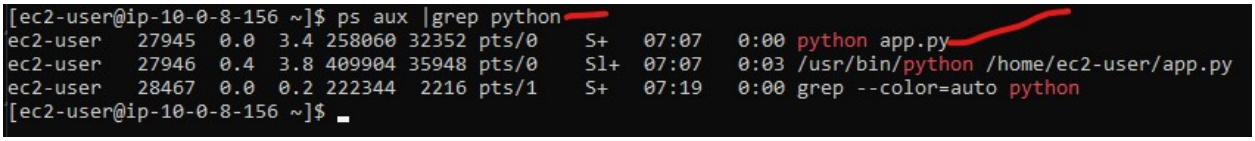


Since the service is running on Interactive session , you can open a new terminal and try to create ,retrive (fetch), updated and delete (CRUD) .

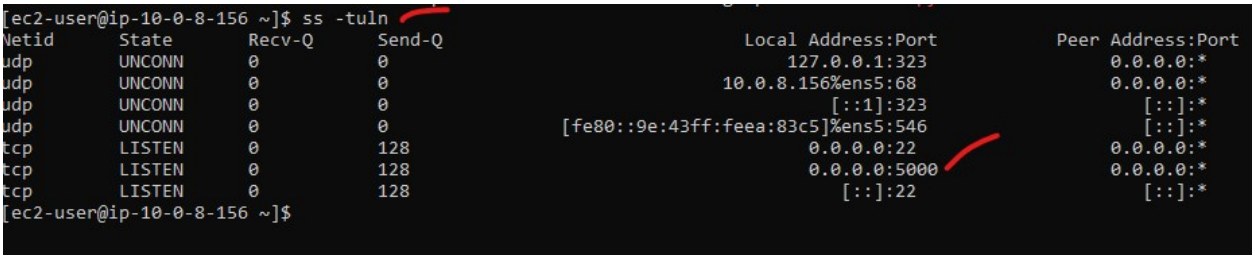
```
curl -X GET http://localhost:5000/users

curl -X POST http://localhost:5000/users/add \
  -H "Content-Type: application/json" \
  -d '{"name":"John Doe","email":"john@example.com"}'

curl -X PUT http://localhost:5000/users/update/1 \
  -H "Content-Type: application/json" \
  -d '{"name":"John Updated","email":"john.updated@example.com"}'

curl -X DELETE http://localhost:5000/users/delete/1
```



You can also check the process **($ps aux |grep python)** and port (**ss -tuln**)



Python Service is running on **port :5000**



Conclusion – Backend server (Bastion host)is communicating with RDS database -1 .

Task Over